ML-Based Cyber Threat Prediction

Domain: Cybersecurity and Machine Learning

A PROJECT PROPOSAL SUBMISSION

By

NAGA GIREESH REDDY (24STUCHH010432)
MANGALI VISHNU DATTA (24STUCHH010201)
DWARAMPUDI NIHAR REDDY (24STUCHH010217)
THALUPURI MANIDEEP SATHYA (24STUCHH010104)

Under the Guidance of

Mr. M. Rama Krishnam Raju

in partial fulfillment for the award of degree
of

BACHELOR OF ENGINEERING

IN
SPECIAL PROJECT

FACULTY OF SCIENCE & TECHNOLOGY
The ICFAI Foundation for Higher Education, Hyderabad



Abstract

Modern networks face a constantly growing volume and sophistication of cyberattacks, including intrusions, denial-of-service attempts, port scans, and malware-driven traffic. Traditional signature-based security tools often fail to detect new or evolving attack patterns, leaving organizations vulnerable to breaches that go unnoticed until significant damage has occurred. This project proposes an ML-Based Cyber Threat Prediction System that uses Machine Learning techniques to analyze network traffic and system logs, identify patterns associated with malicious activity, and predict potential security threats before they escalate. The system examines traffic features such as packet size, connection duration, protocol type, byte counts, and flag patterns to classify network activity as normal or anomalous.

Python, Pandas, NumPy, and Jupyter Notebook are used for data processing and feature engineering, while XGBoost and Scikit-learn are used to train and compare classification models for known attack types, and an Isolation Forest / Autoencoder-based anomaly detection layer is used to catch previously unseen threats. FastAPI can be used for backend services and React.js/Next.js for the security monitoring dashboard. Elasticsearch can be used for storing high-volume traffic logs and prediction history, and Plotly/Chart.js for visualizing threat activity and prediction results. The system provides early threat alerts and supports proactive network defense, helping reduce response time to potential intrusions.

A user-friendly dashboard can be developed to display real-time traffic status, threat probability scores, attack-type classification, and security alerts. The system can flag suspicious connections as they are observed, allowing security teams to investigate and respond before a minor anomaly develops into a full breach.

The proposed solution aims to provide a smart, automated, and data-driven approach to network security monitoring. By predicting cyber threats in advance, the system can help reduce detection time, improve incident response, minimize the impact of attacks, and support a more resilient and intelligent security infrastructure.

Objectives

To develop an AI-powered system for predicting cybersecurity threats in network traffic.

- To collect and process network traffic and system log data.
- To perform data preprocessing, feature selection, and exploratory data analysis.
- To identify patterns and relationships between traffic features and attack types.
- To develop and compare suitable machine learning models for threat prediction.
- To classify network activity into normal and malicious/attack categories.
- To estimate the probability and type of a potential security threat.
- To identify the important features contributing to threat detection.
- To generate early warnings and security alerts for suspicious activity.
- To develop an interactive dashboard for monitoring network health and threat predictions.
- To help reduce detection and response time to cyberattacks.
- To improve organizational security posture through data-driven threat prediction.

Problem Statement

Organizations increasingly rely on interconnected networks that are continuously exposed to cyber threats such as intrusions, denial-of-service attacks, port scanning, and malware communication. Traditional signature-based security systems can only detect previously known attack patterns, leaving networks vulnerable to new or evolving threats. Manual monitoring of network logs is time-consuming, error-prone, and often too slow to prevent damage once an attack has begun.

There is therefore a need for an intelligent system that can analyze network traffic in real time and identify early signs of malicious activity. The proposed ML-Based Cyber Threat Prediction System addresses this problem by applying Machine Learning techniques to network traffic and log data. The system analyzes features such as protocol type, connection duration, packet/byte counts, flag combinations, and traffic frequency to identify patterns associated with cyberattacks.

The proposed system will predict the likelihood of a security threat, classify the type of attack, identify important contributing traffic features, and generate early security alerts. A monitoring dashboard will provide visualization of network activity, prediction results, threat probabilities, and alert history. The system aims to support security teams in making timely decisions, reducing response time to intrusions, and achieving a more proactive and resilient network defense.

Technologies Used

Programming Language — Python

Python is the primary programming language used for developing the ML-Based Cyber Threat Prediction System. It is chosen because of its simplicity, extensive libraries, and strong support for Artificial Intelligence, Machine Learning, and network data analysis. Python is used for data preprocessing, feature engineering, machine learning model development, prediction, backend integration, and communication with the database.

Purpose:

- Data preprocessing
- Machine learning model development
- Prediction and analysis
- Backend API integration

Network Data Handling — Scapy / PyShark

Scapy and PyShark are used to capture, parse, and extract features from network packets and traffic logs. Unlike a generic sensor-data project, threat prediction depends on correctly engineered network features, so these libraries convert raw packets into structured fields (protocol, flags, duration, byte counts) that the ML models can consume.

Purpose:

- Packet capture and parsing
- Traffic feature extraction
- Protocol analysis
- Simulated attack traffic testing

Frontend — React.js

React.js is used to develop the web-based user interface for the system. It provides a fast, responsive, and interactive security dashboard where users can monitor network traffic, view threat scores, check prediction results, and receive real-time alerts.

Purpose:

- Interactive security dashboard
- Real-time traffic monitoring
- Display prediction and alert results
- User-friendly interface
- Real-time visualization

Backend — FastAPI

FastAPI is used as the backend framework to build REST APIs that connect the frontend with the machine learning model. It processes incoming traffic data, retrieves records from the database, sends features to the prediction model, and returns threat classification results quickly.

Purpose:

- REST API development
- Connect frontend and ML model
- Handle prediction requests
- Database communication
- High-performance backend services

Machine Learning Framework — Scikit-learn & XGBoost

Scikit-learn is used for baseline classification models such as Decision Tree, Logistic Regression, and Support Vector Machine (SVM). Because attack traffic is a small minority of total network activity (a highly imbalanced classification problem), XGBoost is used as the primary model — gradient-boosted ensembles consistently outperform plain classifiers on imbalanced intrusion datasets and are the standard choice in published network intrusion detection research.

Purpose:

- Model training
- Threat prediction
- Attack-type classification
- Handling class imbalance
- Model evaluation and comparison

Anomaly Detection — Isolation Forest / Autoencoder (TensorFlow-Keras)

Supervised classifiers can only recognize attack types present in the training data. To also catch unknown or zero-day attacks, an unsupervised anomaly detection layer is added using Isolation Forest and/or a Keras Autoencoder — the model learns what 'normal' traffic looks like and flags any connection that reconstructs poorly or deviates significantly as suspicious, even if it doesn't match a known attack signature.

Purpose:

- Zero-day / unseen attack detection
- Outlier and anomaly scoring
- Complementing supervised classification
- Reducing false negatives on novel attacks

Data Processing Libraries — Pandas and NumPy

Pandas and NumPy are used for cleaning, organizing, preprocessing, and analyzing network traffic data. They help remove missing/inconsistent values, normalize features, perform feature selection, and prepare datasets such as NSL-KDD, CICIDS2017, or UNSW-NB15 for machine learning.

Purpose:

- Data cleaning
- Data preprocessing
- Feature engineering
- Numerical computation
- Dataset manipulation

Data Visualization — Matplotlib and Plotly

Matplotlib and Plotly are used to generate graphs, charts, and visual reports that represent network traffic behavior and prediction results. These visualizations help users understand traffic trends and identify abnormal or suspicious activity.

Purpose:

- Threat trend visualization
- Attack pattern analysis
- Prediction graphs
- Interactive charts
- Performance reports

Database — Elasticsearch

Elasticsearch is used instead of a traditional relational database because network traffic and log data is high-volume, continuously generated, and time-series in nature — exactly the workload Elasticsearch is built for. It is the same storage/search engine used in real-world SIEM (Security Information and Event Management) tools, and lets the system quickly search and aggregate millions of traffic events, prediction records, and alerts.

Purpose:

- Store high-volume traffic logs
- Fast search over historical events
- Store prediction and alert history
- Time-series aggregation and querying
- Scales to real SIEM-style workloads

Development Environment — Jupyter Notebook & Visual Studio Code (VS Code)

Jupyter Notebook is used for exploratory data analysis, machine learning experimentation, and model training. Visual Studio Code is used for application development, debugging, backend implementation, and frontend integration.

Purpose:

- Model experimentation
- Data analysis
- Code development
- Debugging
- Project implementation

Version Control — Git & GitHub

Git and GitHub are used to manage source code, track project changes, and collaborate efficiently during development. They help maintain different versions of the project and simplify teamwork.

Purpose:

- Source code management
- Version control
- Collaboration
- Backup
- Project tracking

Real-Time Data Streaming — Apache Kafka (Optional)

Apache Kafka can optionally be used to stream live network traffic events into the prediction pipeline instead of processing data in static batches. This is included as a stretch goal to demonstrate how the system could scale to near-real-time detection in a production network environment.

Purpose:

- Live traffic ingestion
- Decoupling capture from prediction
- Near-real-time alerting
- Scalability for production-scale traffic

Deployment — Docker (Optional)

Docker is used to package the entire application, including the frontend, backend, machine learning model, and database configuration, into containers. This makes deployment easier and ensures the application runs consistently across different environments.

Purpose:

- Application deployment
- Containerization
- Easy installation
- Environment consistency
- Scalability

Expected Outcomes

- A trained and evaluated threat classification model capable of distinguishing normal traffic from multiple attack categories.
- An anomaly detection layer capable of flagging previously unseen (zero-day) attack patterns.
- A comparative performance report across baseline and ensemble models (accuracy, precision, recall, F1-score).
- A functional web dashboard displaying live/near-live traffic status, threat scores, and alerts.
- A documented, reproducible pipeline from raw traffic data to prediction and alerting.